

Pivot Consolidation Table

Function-by-Function Code Walkthrough (Latest Version)

A Jupyter-notebook-style deep dive into `src/visual.ts` — covering the latest additions: the 3-level header (big group → Group → column), per-Group independent Total-column toggles, and the auto-computed Total column at the big-group level. Every function is explained with Python-style '#' comments annotating the TypeScript logic line by line, plus flow diagrams for the trickiest algorithms.

1. Overall render pipeline

Like every custom visual, everything starts from **update()**. The Pivot version adds one major extra stage compared to Fixed Groups: instead of columns coming straight from fixed Group 1-6 fields, they come from Power BI's automatic 'grouped by category' mechanism, and are then re-organized into 'blocks' based on which big group (if any) each Group 1-6 bucket belongs to.

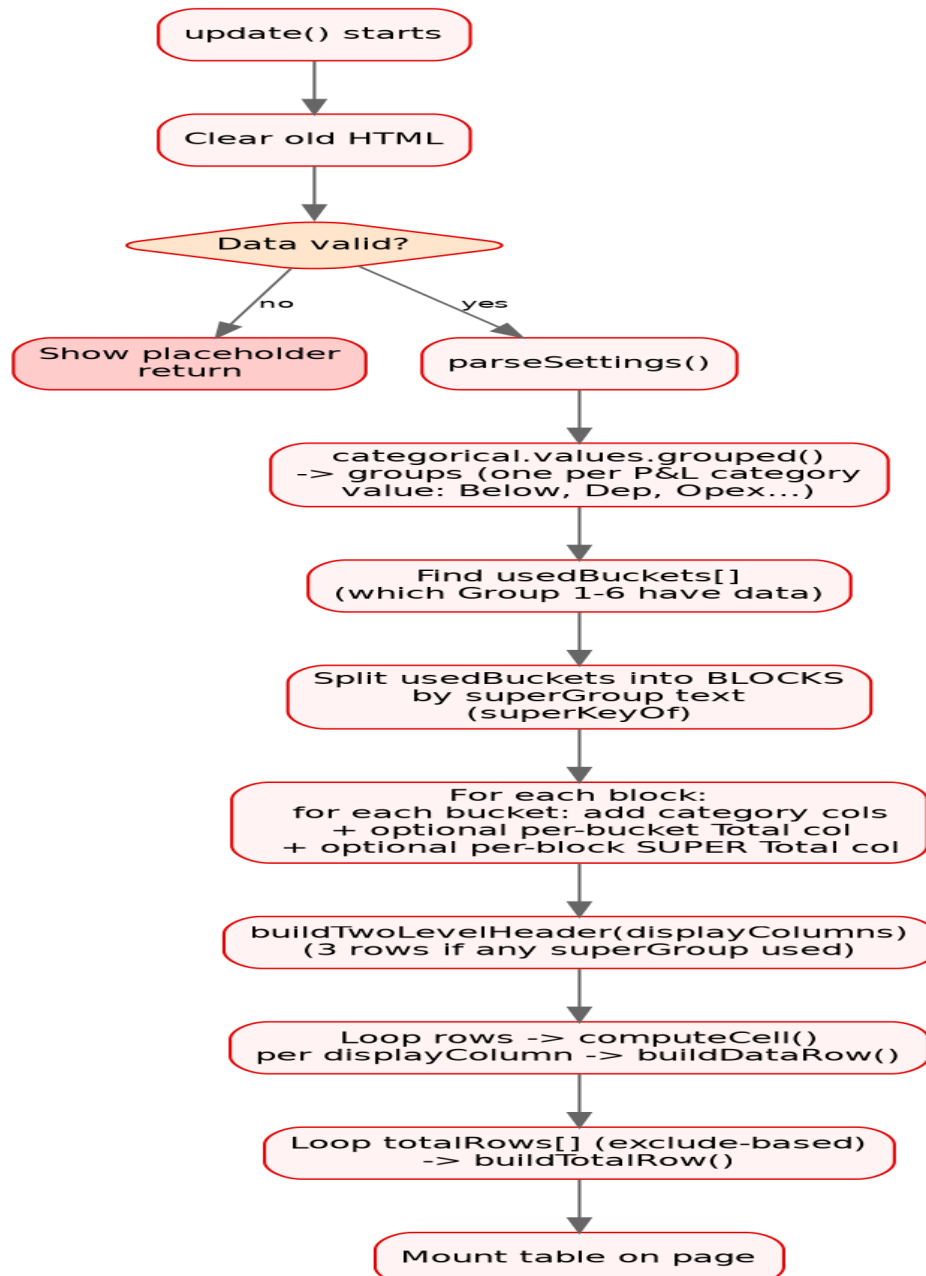


Figure 1 — the full `update()` render pipeline for Pivot Consolidation Table

2. Module-level helper functions

Small pure functions, declared outside the class, reused throughout the render pipeline.

2.1 getDefaultSettings()

```
function getDefaultSettings(): ISettings
```

Factory defaults for every configurable property, including the two NEW settings that power this version: groupTotalEnabled (per-bucket Total toggle) and superGroups (big-group assignment per bucket).

```
def get_default_settings():
    return {
        "negativeColor": "#E20000",
        "headerColor": "#E20000",
        "totalRowColor": "#F0F0F0",
        "fontSize": 13,
        "defaultExpanded": True,
        "groupGapWidth": 8,
        "showToggleIcon": True,
        "boldMainRows": True,
        "boldChildNumbers": False,
        "groupTotalEnabled": [True] * 6,          # NEW: every Group's own Total column sta
rts ON
        "totalSubColumnLabel": "Total",
        "showSuperTotalColumn": True,             # NEW: big-group Total column starts ON
        "superTotalColumnLabel": "Total",
        "bucketLabels": [""] * 6,
        "superGroups": [""] * 6,                 # NEW: no big group assigned to any buck
et yet
        "relabelMap": {},
        "totalRows": [
            {"enabled": False, "label": "Total", "exclude": []},
            {"enabled": False, "label": "Total", "exclude": []},
            {"enabled": False, "label": "Total", "exclude": []},
        ],
        "descTwoLines": False,
        "descLine1": "Description",
        "descLine2": "",
        "descSeparatorSize": 1,
    }
```

Step-by-step:

- groupTotalEnabled = [True]*6 means: out of the box, every Group's own Total column is visible — the user has to explicitly switch one OFF, not explicitly turn each ON.
- superGroups = [""]*6 (all blank) is exactly what keeps the header at 2 levels by default — the 3rd row only appears once at least one bucket gets a big-group name.

2.2 parseRelabelMap / parseRowList / relabel / formatNumber

(module-level helpers, unchanged from the Fixed Groups visual)

These four functions are byte-for-byte identical to the Fixed Groups version: parsing the 'key=value;key2=value2' rename string, parsing a comma-separated exclude list into a lower-cased array, looking up a renamed label, and formatting numbers with thousands separators and parenthesized negatives.

```
def format_number(value, decimals=0):
    if value is None or is_nan(value):
        return ""
    abs_value = abs(value)
    formatted = f"{abs_value:,.{decimals}f}"
    return f"({formatted})" if value < 0 else formatted

def relabel(rename_map, key):
    return rename_map[key] if key in rename_map else key

def parse_row_list(text):
    if not text:
        return []
    return [p.strip().lower() for p in text.split(",") if p.strip()]
```

Step-by-step:

- Reusing identical logic across both visuals is intentional — it guarantees the 'Rename columns' syntax and the exclude-list syntax behave IDENTICALLY whichever chart you're configuring, so there's only one convention to remember.
-

3. Building displayColumns: the heart of this version

This is entirely new compared to Fixed Groups. Instead of one column per physical field, Power BI's grouped() call already pivots the data by category (Below/Dep/Opex). The visual's job is to re-organize those pivoted values into 'blocks' — runs of Group 1-6 buckets that share the same big-group assignment — and inject the auto-computed Total columns at both the per-Group and per-big-group levels.

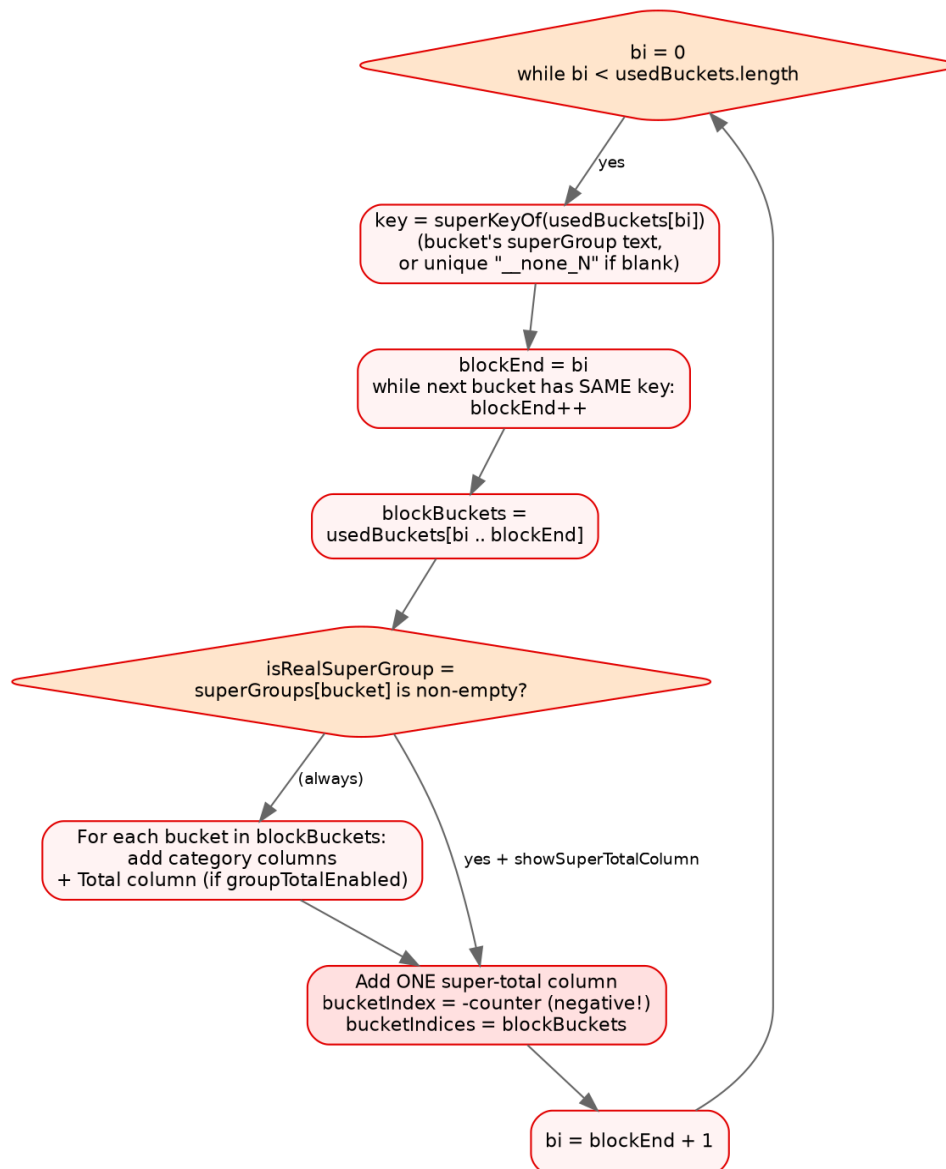


Figure 2 — the block-grouping loop that builds `displayColumns[]`

3.1 superKeyOf(bucketIdx) — the grouping key

```
const superKeyOf = (bucketIdx: number) => string
```

A tiny inline function (declared with `const`, not `function`) that returns the 'grouping key' for one bucket: its big-group text if it has one, or a guaranteed-unique placeholder if it doesn't.

```
def super_key_of(bucket_idx):
    sg = settings.superGroups[bucket_idx] or ""
    if len(sg) > 0:
        return sg                                # e.g. "Act Vs. RF" -> buckets sharing this MERG
E
    else:
        return f"__none_{bucket_idx}"          # unique per bucket -> NEVER merges with a neigh
bour
```

Step-by-step:

- This one line is what prevents a subtle bug: if two buckets both have NO big-group assigned, they must NOT be silently merged together just because they're both blank. Giving each blank bucket a unique key (via its own index) guarantees that.

3.2 The block-splitting while loop

```
let bi = 0; while (bi < usedBuckets.length) { ... }
```

Walks `usedBuckets` once, and for each position, grows a 'block' forward as long as the next bucket shares the same `superKeyOf(...)` — classic run-length grouping, exactly like the header-building algorithm, but applied one level higher (grouping BUCKETS instead of grouping COLUMNS).

```
bi = 0
while bi < len(used_buckets):
    key = super_key_of(used_buckets[bi])
    block_end = bi
    # keep growing the block while the NEXT bucket shares the same key
    while block_end + 1 < len(used_buckets) and super_key_of(used_buckets[block_end+1]) =
= key:
        block_end += 1

    block_buckets = used_buckets[bi : block_end + 1]          # every bucket in this block
    is_real_super_group = len(settings.superGroups[used_buckets[bi]] or "") > 0

    for bucket_idx in block_buckets:
        super_group_label = settings.superGroups[bucket_idx] or ""
        for cat_idx, cat_name in enumerate(category_order):    # Below, Dep, Opex..
            .
            display_columns.append({
                "bucketIndex": bucket_idx,
                "categoryKey": cat_name,
                "label": relabel(settings.relabelMap, cat_name),
                "superGroup": super_group_label,
                "isGroupStart": cat_idx == 0,
                "isTotalColumn": False,
                "isSuperTotalColumn": False,
            })
        if settings.groupTotalEnabled[bucket_idx]:              # PER-GROUP toggle
```

```

check
    display_columns.append({
        "bucketIndex": bucket_idx,
        "categoryKey": TOTAL_KEY,
        "label": settings.totalSubColumnLabel,
        "superGroup": super_group_label,
        "isGroupStart": len(category_order) == 0,
        "isTotalColumn": True,
        "isSuperTotalColumn": False,
    })

if is_real_super_group and settings.showSuperTotalColumn:
    super_total_counter += 1
    display_columns.append({
        "bucketIndex": -super_total_counter,          # NEGATIVE = "not a real bucket
        "bucketIndices": block_buckets,               # remembers ALL buckets to sum
        "categoryKey": TOTAL_KEY,
        "label": settings.superTotalColumnLabel,
        "superGroup": settings.superGroups[used_buckets[bi]] or "",
        "isGroupStart": True,
        "isTotalColumn": True,
        "isSuperTotalColumn": True,
    })

    bi = block_end + 1                                # jump straight to the next block's first bucket
t

```

Step-by-step:

- Notice the per-Group Total check happens INSIDE the bucket loop (settings.groupTotalEnabled[bucket_idx]) — each bucket decides independently whether it gets a Total column.
 - The super-total column is only added ONCE per block (outside the bucket loop), and only if the block actually represents a real named big-group AND the global showSuperTotalColumn switch is on.
 - bucketIndex is set to a NEGATIVE number (-super_total_counter) purely as a tagging trick: every real bucket is 0-5, so any negative bucketIndex unambiguously means 'this is a synthetic super-total column, not a real data column'.
-

4. computeCell(): three calculation paths

Every single number displayed in the table — whether it's a plain category value, a per-Group Total, or a big-group Total — passes through this one function. It branches into exactly three cases, checked in a specific order.

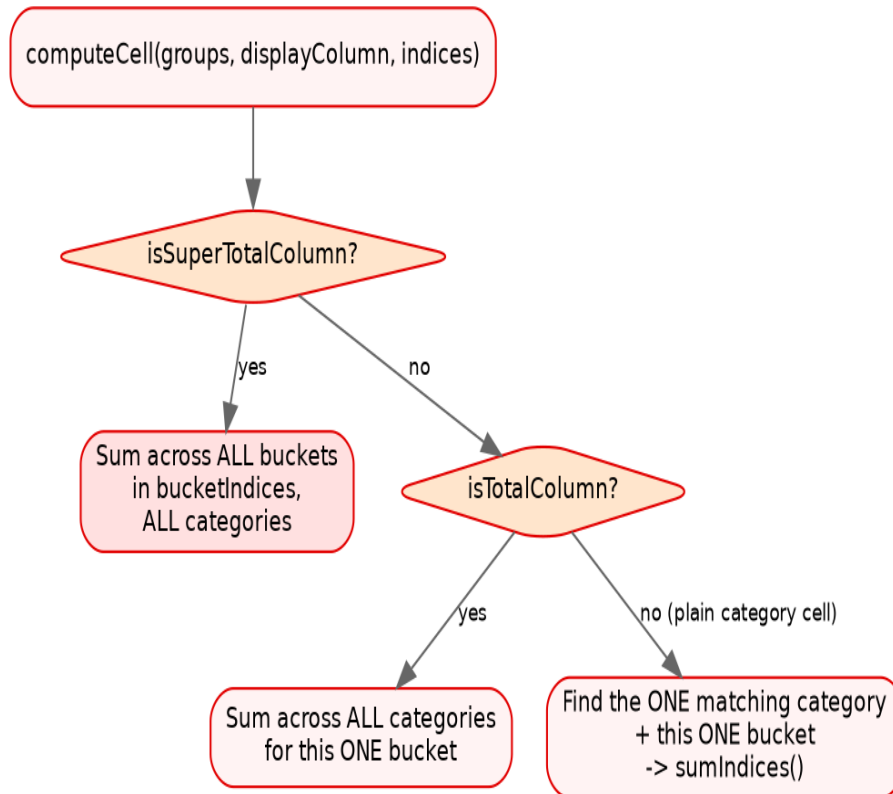


Figure 3 — the three branches inside `computeCell()`

4.1 `computeCell(groups, displayColumn, indices)`

```
private computeCell(groups, dc: IDisplayColumn, indices: number[]): number
```

Given one 'display column' description and a list of underlying data-row indices (e.g. all rows belonging to one Department), returns the single aggregated number to show in that cell.

```
def compute_cell(groups, dc, indices):
    # ----- BRANCH 1: big-group Total column -----
    if dc["isSuperTotalColumn"]:
        total = 0
        for bucket_idx in dc["bucketIndices"]:
            # every bucket under this big group
            for g in groups:
                # every category (Below, Dep, Opex)
                col = find_bucket_column(g, bucket_idx)
                if col:
                    total += sum_indices(col, indices)
        return total

    # ----- BRANCH 2: per-Group Total column -----
    if dc["isTotalColumn"]:
        total = 0
```



```

    for g in groups:                                # every category, ONE bucket
only        col = find_bucket_column(g, dc["bucketIndex"])
            if col:
                total += sum_indices(col, indices)
            return total

# ----- BRANCH 3: plain category cell (e.g. "Below") -----
group = next((g for g in groups if str(g.name or "") == dc["categoryKey"]), None)
if not group:
    return 0
col = find_bucket_column(group, dc["bucketIndex"])
if not col:
    return 0
return sum_indices(col, indices)

```

Step-by-step:

- Branch 1 is checked FIRST, because a super-total column also has `isTotalColumn = True` — if the order were reversed, branch 2 would incorrectly catch it and only sum one bucket instead of the whole block.
- Branch 2 sums across every CATEGORY for one fixed bucket (this is a 'Group Total').
- Branch 3 is the simplest and most common case: no summing across anything, just one specific (category, bucket) pair looked up directly.

4.2 findBucketColumn(group, bucketIndex)

`private findBucketColumn(group, bucketIndex: number): DataViewValueColumn | undefined`

Inside one category's group of columns, finds the specific physical column that came from a given Group 1-6 bucket, by inspecting its Data-pane role tag.

```

def find_bucket_column(group, bucket_index):
    role = BUCKET_ROLES[bucket_index]                # e.g. bucket_index=2 -> "bucket3"
    return next((c for c in group.values if c.source.roles and c.source.roles.get(role)),
None)

```

Step-by-step:

- Used by every branch of `computeCell()` — it's the low-level lookup that all three paths share.

4.3 sumIndices(col, indices)

`private sumIndices(col, indices: number[]): number`

Identical in spirit to the Fixed Groups version: sums a physical column's values across a given set of row indices, skipping blanks safely.

```

def sum_indices(col, indices):
    total = 0
    for i in indices:

```

```
    v = col.values[i]
    if v is not None and not is_nan(v):
        total += v
return total
```

5. Building the (up to) 3-level header

The header now conditionally renders either 2 or 3 <tr> rows, decided once at the top by scanning whether ANY column carries a non-empty superGroup.

5.1 buildDescriptionCell(hasSuperRow)

```
private buildDescriptionCell(hasSuperRow: boolean): HTMLTableCellElement
```

Same top-left cell as before, but its `rowSpan` is now dynamic: 3 rows tall when a big-group row exists, 2 rows tall otherwise — so it always spans exactly the full header height.

```
def build_description_cell(has_super_row):
    cell = create_element("td")
    cell.rowSpan = 3 if has_super_row else 2          # <-- the only real change vs Fixed
    Groups
    cell.className = "pc-desc-header"
    # ... two-line / one-line text logic identical to before
    return cell
```

5.2 buildTwoLevelHeader(columns) — now up to 3 rows

```
private buildTwoLevelHeader(columns: IDisplayColumn[]): HTMLTableSectionElement
```

Builds the optional big-group row, the always-present Group-name row, and the always-present category row — each using the same run-length `colSpan`-merging technique, just keyed differently.

```
def build_two_level_header(columns):
    thead = create_element("thead")
    has_super_row = any(c["superGroup"] for c in columns)

    # ----- ROW 0 (conditional): big-group names -----
    if has_super_row:
        super_row = create_element("tr")
        super_row.append(build_description_cell(True))

        def super_key(c):
            return c["superGroup"] if c["superGroup"] else f'__none_{c["bucketIndex"]}'

        i = 0
        while i < len(columns):
            key = super_key(columns[i])
            span = 1
            while i + span < len(columns) and super_key(columns[i+span]) == key:
                span += 1
            cell = create_element("td")
            cell.colSpan = span
            if i > 0:
                cell.classList.add("pc-group-gap")
            cell.textContent = columns[i]["superGroup"] or ""
            super_row.append(cell)
            i += span
        thead.append(super_row)

    # ----- ROW 1: Group names (always present) -----
```

```

top_row = create_element("tr")
if not has_super_row:
    top_row.append(build_description_cell(False))    # only placed here if row 0 is
absent

i = 0
while i < len(columns):
    bucket_idx = columns[i]["bucketIndex"]
    span = 1
    while i + span < len(columns) and columns[i+span]["bucketIndex"] == bucket_idx:
        span += 1
    cell = create_element("td")
    cell.colSpan = span
    if i > 0:
        cell.classList.add("pc-group-gap")
    if columns[i]["isSuperTotalColumn"]:
        cell.classList.add("pc-total-col")
        cell.textContent = columns[i]["label"]    # e.g. "Total" -- own single
cell
    else:
        label = settings.bucketLabels[bucket_idx]
        cell.textContent = label if label else f"Group {bucket_idx + 1}"
    top_row.append(cell)
    i += span
thead.append(top_row)

# ----- ROW 2: category / Total sub-columns (always present) -----
sub_row = create_element("tr")
for idx, c in enumerate(columns):
    cell = create_element("td")
    if c["isGroupStart"] and idx > 0:
        cell.classList.add("pc-group-gap")
    if c["isTotalColumn"]:
        cell.classList.add("pc-total-col")
    cell.textContent = c["label"]
    sub_row.append(cell)
thead.append(sub_row)

return thead

```

Step-by-step:

- Row 0's run-length grouping uses `super_key()` (text-based); Row 1's uses `bucketIndex` directly (number-based) — two independent groupings layered on the SAME underlying columns array.
 - The description cell is appended to whichever row is drawn FIRST (row 0 if present, otherwise row 1) — this if/else guard is what stops it being duplicated.
 - A super-total column's `bucketIndex` being negative and unique means it automatically gets its OWN single-width cell in Row 1 (since no neighbouring column shares that exact negative number), which is exactly the desired visual: the super-total column stands alone under the big-group row.
-

6. Row rendering & the settings round-trip

`buildDataRow()`, `buildTotalRow()`, `appendToggleAndLabel()`, and `applyThemeVars()` are functionally identical to the Fixed Groups visual (same expand/collapse mechanism, same bold/color rules, same CSS-variable approach) — see the Fixed Groups deep-dive PDF for their full line-by-line breakdown. The interesting differences live entirely in `parseSettings()` and `enumerateObjectInstances()`, which now also handle the two new Format-pane sections.

6.1 `parseSettings()` — the two new sections

```
private parseSettings(dataView: DataView): ISettings
```

In addition to the usual `dataPoint` / `tableFormat` / `bucketLabels` / `columnLabels` / `totalRows` / `descriptionHeader` sections, two more are now read: `groupTotals` and `superGroups`.

```
# ... same defensive pattern as before, then:

gt = objects.get("groupTotals")
if gt:
    settings.groupTotalEnabled = [
        gt.get(f"bucket{n}ShowTotal", True) if isinstance(gt.get(f"bucket{n}ShowTotal"),
bool) else True
        for n in range(1, 7)
    ]
    # default True if the key is missing entirely

sg = objects.get("superGroups")
if sg:
    settings.superGroups = [
        sg.get(f"bucket{n}Super", "") if isinstance(sg.get(f"bucket{n}Super"), str) else
""
        for n in range(1, 7)
    ]
```

Step-by-step:

- `groupTotalEnabled` defaults to `True` per-bucket even inside this parser (not just in `getDefaultSettings`) — so a bucket added AFTER the visual was first configured still starts with its `Total` column visible, rather than inheriting some stale `False`.

6.2 `enumerateObjectInstances()` — the two new sections

```
public enumerateObjectInstances(options): VisualObjectInstanceEnumeration
```

Mirrors `parseSettings()`: whenever the user opens the 'Group Total column' or 'Big group above' Format-pane sections, these blocks report back the 6 current values each.

```
if object_name == "groupTotals":
    props = {}
    for idx, enabled in enumerate(settings.groupTotalEnabled):
        props[f"bucket{idx+1}ShowTotal"] = enabled
    instances.append({"objectName": object_name, "properties": props, "selector": None})

if object_name == "superGroups":
```

```
props = {}
for idx, label in enumerate(settings.superGroups):
    props[f"bucket{idx+1}Super"] = label
instances.append({"objectName": object_name, "properties": props, "selector": None})
```

Step-by-step:

- Same idx+1 pattern used everywhere else in this codebase (Format pane properties are 1-indexed, arrays are 0-indexed).
-

End of walkthrough. Together with the Fixed Groups deep-dive PDF (which covers the shared row-rendering, expand/collapse, and theming functions in full detail), this document completes the picture of every function in the Pivot Consolidation Table's latest version: the block-grouping algorithm, the three-branch computeCell(), the up-to-3-level header builder, and the extended settings round-trip.